Porting the Autoware Safety Island onto nano-ros: a Reference-Consumer Migration Study

NEWSLab, National Taiwan University

Technical report for the Autoware Foundation — 2026-08-20

Abstract. The Autoware Safety Island (ASI) runs Autoware's trajectory follower — MPC lateral and PID longitudinal control — as a standalone application on a safety-class Arm processor, exchanging commands with a full Autoware stack over DDS. This report documents the migration of ASI's Zephyr targets from a hand-glued CycloneDDS port onto *nano-ros*, a lightweight `no_std` ROS 2 client library for embedded RTOSes, and the methodology used: ASI serves as nano-ros's *canonical reference consumer*, so every integration failure ("wall") is either fixed upstream or captured as a tracked issue rather than papered over downstream. We describe the consumption model (git submodule in lockstep with a `west` manifest pin, board-crate provisioning, generated entry from a resolved system model), a modernization step that advanced the middleware pin by roughly 2,400 commits in one pass, the eleven walls that step surfaced with their root causes, and the resulting validation: six firmware build modes, a five-phase emulator runtime suite, and a closed-loop demonstration against an unmodified ROS 2 Humble Autoware stack at control rates matching the pre-migration baseline. Wire-level interoperability with `rmw_cyclonedds` is preserved by construction. We conclude with lessons transferable to other middleware migrations in the Autoware ecosystem.

1 Introduction

Safety architectures for autonomous driving increasingly separate a *safety island* — a minimal, certifiable control path on a lockstep-capable processor — from the performance-oriented compute domain running the full autonomy stack. The Autoware Safety Island project realizes this pattern for Autoware: the trajectory follower (MPC lateral controller and PID longitudinal controller, vendored unmodified from Autoware) executes on an Arm Cortex-R82 class target under Zephyr RTOS, receives the planned trajectory and vehicle kinematic state over DDS, and returns `autoware_control_msgs/Control` commands to the primary stack (optionally mirrored onto CAN).

The original port carried its own middleware glue: a vendored CycloneDDS tree, hand-written IDL tooling on the host, and a bespoke node/executor shim. That glue is exactly the part a safety argument least wants to own. nano-ros — a lightweight ROS 2 client for embedded RTOSes (Zephyr, FreeRTOS, NuttX, ThreadX) with interchangeable RMW backends (CycloneDDS, zenoh-pico, Micro-XRCE) — offers the same wire compatibility with a maintained, testable, upstreamable middleware layer. This report documents how ASI's Zephyr targets were moved onto nano-ros, and what the process taught both projects.

Two properties make this migration worth reporting beyond its immediate result. First, it was run as a *reference-consumer* engagement: ASI is nano-ros's named consumer archetype, and the migration contract required every consumer-surfaced defect to be intaken upstream (nano-ros's phase-292 intake and issue tracker) rather than worked around silently. Second, the final modernization step was deliberately large — a single pin advance of $\approx 2,400$ upstream commits — which stress-tests how well a middleware's consumer surface holds together across months of internal change.

2 System overview

2.1 The safety island application

ASI bundles the relevant Autoware components (trajectory follower node, MPC and PID controllers, interpolation and vehicle-info utilities) unmodified under an adapter layer. Platform specifics sit behind `common/` interfaces (node, clock, logger, CAN, network). Production target is the Arm FVP_BaseR_AEMv8R fixed virtual platform (Cortex-R82, SMP-4) and the NXP S32Z270 board; a FreeRTOS POSIX simulator serves development. The controller subscribes to five inputs (trajectory, kinematic state, acceleration, steering status, operation mode), runs a 30 ms control timer, and publishes control commands, all over DDS domain boundaries bridged to the primary Autoware domain.

2.2 nano-ros consumption model

The migration adopted nano-ros's *board-crate consumer* shape, which the nano-ros book documents with ASI as the archetype:

- **Checkout.** nano-ros is a git submodule at `modules/nros`, pinned in lockstep with the `west` manifest revision (`import: false`; ASI keeps manifest authority). `west update` adopts the submodule checkout in place.
- **Provisioning.** The in-tree `nros` CLI provisions the consumer's Zephyr tree from the board's declared contract: the RMW source (a CycloneDDS 0.10.5 fork pinned to the version ROS 2 Humble ships), a Zephyr-line patch set, Rust cross-targets, and host tools (the CycloneDDS `idlc` from an SDK store) — one command per concern, no hand-maintained forks downstream.
- **Board import.** One CMake call, `nano_ros_use_board(fvp-aemv8r-smp)`, placed before `find_package(Zephyr)`, layers in the board id, Kconfig fragments, device-tree overlay, default RMW, and the runner hint.
- **Messages.** The ten vendored ROS interface packages (`geometry_msgs`, `autoware_control_msgs`, …) are declared once; nano-ros codegen emits C++ value types with fixed-capacity sequences/strings (no heap growth on the receive path), per-package Rust FFI crates, and CycloneDDS topic descriptors whose type names match `rmw_cyclonedds`'s on-wire naming.
- **Entry.** The bootable image is *generated*: the launch topology and parameters are authored declaratively (`system.toml` + launch XML), resolved by `play_launch` into a system model, and baked by `nano_ros_add_executable(... MODEL ... TYPED)` into a Zephyr entry that constructs the controller component with its 74 launch parameters. The controller itself is an `nros::ComponentNode` (rclcpp-faithful surface: `declare_parameter`, member-function-pointer subscriptions and timers, value-typed publishers), registered under nano-ros's package/class identity rule.

This shape eliminates the three artifacts a hand-glued port accumulates: duplicated board configuration, hardcoded launch/parameter wiring, and a private middleware fork.

3 Migration method: the reference-consumer contract

The port proceeded in phases (tracked in the repository's phase-1 through phase-3 roadmap documents): an initial build-system and codegen spike, a node-adapter phase introducing a polling shim over nros-cpp, migration of the controller onto `ComponentNode`, adoption of workspace mode and the canonical CMake verbs, and finally runtime proof on the FVP. Each phase ended with the same discipline:

1. Attempt the step exactly as nano-ros's user documentation prescribes.
2. Record every failure as a numbered *wall* with a minimal reproduction.
3. File the wall upstream (nano-ros intake/issue) when the cause is upstream; fix it downstream only when it is genuinely consumer-side.

4. Re-verify from a clean workspace before declaring the phase done.

An earlier pin advance (July 2026) surfaced eight walls, all of which were fixed on nano-ros main within the same cycle — among them Zephyr multicast-join in the CycloneDDS fork, a mutex-pool exhaustion under SMP, and per-package FFI generation. The consumer benefit of that discipline showed up in this modernization: the follow-on jump of $\approx 2,400$ commits produced *no* regression in any area an earlier wall had hardened.

4 The 2026-08-20 modernization

The step advanced the nano-ros pin from `7dfe4fe4e` (2026-07-21) to `eace28852` (2026-08-20) and adopted the submodule layout described above. Eleven walls were logged; none required changes to the vendored Autoware components, and application logic was untouched throughout.

#	Symptom	Root cause	Disposition
1	`nros setup board` rejects the board	CLI resolves the retired per-board crate path; bundle boards moved in an upstream refactor	Upstream issue #0729; steps inlined downstream
2	Configure fails: host `idlc` not found	Host CycloneDDS tooling moved to the CLI-managed SDK store	Provision via `nros setup <board> --rmw cyclonedds`
3	Board-facts rung silently absent	Facts resolver needs the nano-ros checkout path outside its own tree	`NROS_REPO_DIR` exported; residual is #0729′s class
4	`nros-cpp` fails to compile (E0599)	Embedded C++ CycloneDDS feature set omits `alloc` while an error-mapper arm references an alloc-gated variant; masked upstream by `native_sim`'s `std`	Upstream issue #0730; self-retiring downstream patch
5	Test programs: missing headers, unknown type names	Flat idlc-name compatibility layer retired; typed C++ API is now the only publisher/subscriber surface	Tests migrated onto the dual-mode message umbrella
6	`CAN_FILTER_DATA` undeclared	Zephyr 3.7 removed the flag; the CAN test had never been rebuilt on 3.7	Standard-ID filter is `flags = 0`
7	Entry panic policy changed	Upstream default moved from halt to the platform's `k_panic()`	Accepted; loud fatal is preferable for a safety island
8	Sizing knobs went live	Previously inert Kconfig knobs are now forwarded; environment overrides Kconfig	Audited: build-time exports still authoritative
9	Boot markers never printed	Markers lived in the retired imperative `main.cpp`; the generated entry prints no application banner	Component constructor now owns the markers
10	Tests: `create_node` returns `NOT_INIT`	Entry-less test images never initialized the runtime	Shim performs one-time `nros::init()`
11	Test hangs on node stop	`pthread_cancel` is deferred-only on Zephyr's POSIX layer; the poll loop has no cancellation point	Cooperative stop flag replaces cancel

Table 1: Wall ledger for the $\approx 2,400$-commit pin advance. Full details in the repository's phase-3 roadmap document.

Three observations generalize:

Latent-validation debt surfaces on middleware moves. Walls 6, 9, 10 and 11 were not caused by the new pin — they were pre-existing gaps (a Zephyr 3.7 API removal, boot markers owned by retired code, tests relying on an implicit initialization, a cancellation idiom that never worked on this RTOS) that the first *complete* runtime re-validation exposed. A migration is only as trustworthy as the test surface actually re-run on it.

Coverage gaps are pairwise, not per-axis. Wall 4′s failing coordinate is the *combination* (embedded Zephyr × C++ × CycloneDDS); every individual axis value was covered upstream, but the pairing was not. This is the classic covering-array argument, and it argues for pairwise CI lanes on any matrixed middleware.

Consumer workarounds should be designed to die. The two downstream workarounds (walls 1 and 4) are idempotent scripts that detect the upstream fix and retire themselves, keeping the consumer's fork-pressure at zero.

5 Validation

Build. All six firmware build modes compile and link from one branch: the full controller image, unit test, DDS loopback, CAN output test, and the DDS publisher/subscriber pair (RAM footprint of the full image: 9.7 MB of 128 MB).

Runtime. The repository's five-phase CI script was run against Arm FVP_BaseR_AEMv8R 11.31.28: (1) full controller boot to its liveness markers; (2) on-target unit suite — parameters, timers, thread lifecycle, DDS pub/sub round-trip, clock conversions — to `All Tests Passed`; (3) DDS loopback; (4) CAN output over Zephyr's CAN loopback driver; (5) TAP-network image build. All phases pass.

Closed loop. The demonstration stack — unmodified Autoware (`ghcr.io/autowarefoundation/autoware:universe-20250207`, ROS 2 Humble, `rmw_cyclonedds`) with its planning simulator, a DDS domain bridge, and the island on the FVP attached via a TAP interface on DDS domain 2 — was brought up headless; the ego pose and a goal were seeded from the command line. The planner streams the trajectory at 10 Hz through the bridge; the island's MPC engages (and, from the spawn position, correctly commands an emergency stop on excessive tracking error) and publishes `/control/trajectory_follower/control_cmd` back onto domain 1 at 19–22 Hz, matching the pre-modernization baseline (~19 Hz). A single command, `scripts/run-tap-demo.sh`, now reproduces this end-to-end.

ROS interoperability. Interoperability with stock ROS 2 is exercised in both directions by the closed loop itself: the island deserializes the planner's trajectory, odometry, acceleration and operation-mode messages, and the Humble side subscribes to and field-level-decodes the island's `Control` messages (verified with `ros2 topic echo`; CDR layout and `dds__`-style type names match `rmw_cyclonedds` by construction, since nano-ros pins its CycloneDDS fork to the release ROS ships). No bridge-side type adaptation exists anywhere in the path.

6 Lessons for the ecosystem

1. **A reference consumer is cheap leverage for a middleware project.** Eleven walls in one step, two of them genuine upstream bugs invisible to the project's own CI, at the cost of a disciplined ledger.

2. **Pin in lockstep, and make the pin visible.** A submodule pointer plus a manifest revision that must agree gives reviewers one diffable fact and makes "what moved?" answerable by `git log` on the submodule.
3. **Generated entries beat imperative boot code** — but application-level observability (liveness markers, boot status) must then be owned by application components, not assumed from retired scaffolding.
4. **Treat provisioning as data.** Everything the consumer's host needs (toolchains, RMW sources, patch sets) is declared in nano-ros's SDK index and applied by idempotent commands; the consumer's bootstrap is thin and survives upstream refactors that would break hand-rolled scripts.
5. **Re-run everything on every move.** The walls that cost the most wall-clock time were the ones hiding behind validation that had silently stopped running.

7 Status and future work

The Zephyr FVP target is fully migrated and validated on the modern nano-ros pin. Open items: S32Z270 board parity (its upstream board bundle is pending), moving the FreeRTOS targets onto nano-ros's platform layer (retiring the vendored CycloneDDS entirely), adopting the launch-resolved entry spelling end-to-end, a long-duration soak against the Phase-2 real-run checkpoint scenario, and upstreaming resolutions for issues #0729/#0730.

Sources: `docs/roadmap/phase-3-modern-nano-ros-migration.md` (wall ledger, runbook), `docs/roadmap/phase-1*.md`, `docs/design/workspace_mode.rst`, nano-ros book chapter *Importing a Board Crate*, nano-ros issues #0729/#0730. Repository: `github.com/newslabntu/autoware-safety-island`, branch `nano-ros`.